

Flexible Load Balancing

Microservices Architecture with Spring Boot & Docker

Group 12 | Microservices Course | 2026

Spring Boot

Eureka

Docker

Load Balancer

Agenda

01

What is Load Balancing?

Concept & motivation

02

System Architecture

3 microservices overview

03

Technologies Used

Spring Cloud, Docker, Eureka

04

Implementation

Code & configuration details

05

Live Demo Results

Load distribution in action

What is Load Balancing?

Load Balancing distributes incoming network traffic across multiple servers or service instances, ensuring no single server becomes a bottleneck.

Round Robin

Requests sent to each instance in order, equally

Random

Random instance selected per request

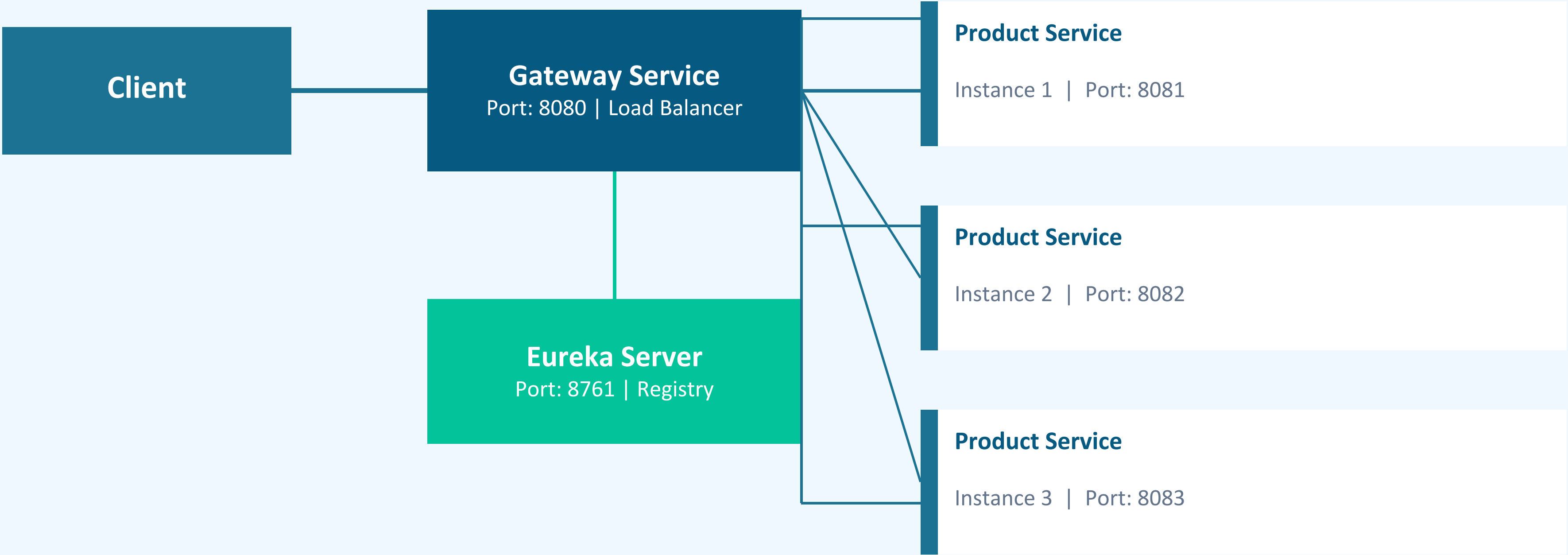
Least Connection

Routes to instance with fewest active connections

Weighted

More traffic to higher-capacity instances

System Architecture



Technologies Used

Java 21

LTS programming language

Spring Boot 3.2

Microservice framework

Spring Cloud Gateway

API routing & load balancing

Netflix Eureka

Service discovery & registry

Docker

Containerization platform

Docker Compose

Multi-container orchestration

Implementation Highlights

Eureka Server

```
@EnableEurekaServer
```

```
register-with-eureka: false
```

```
fetch-registry: false
```

Product Service

```
@EnableDiscoveryClient
```

```
GET /products → returns instance ID + port
```

```
3 instances: ports 8081, 8082, 8083
```

Gateway Service

```
uri: lb://product-service
```

```
Path=/api/products/**
```

```
Filter: StripPrefix=1
```

Docker Compose

```
5 containers in bridge network
```

```
restart: on-failure policy
```

```
docker-compose up --build
```

All services containerized with eclipse-temurin:21-jdk-alpine Docker image for minimal footprint

Live Demo Results

Eureka Dashboard (localhost:8761) — All 4 instances registered and UP

Request 1 →

product-instance-1

Port 8081

Request 2 →

product-instance-3

Port 8083

Request 3 →

product-instance-2

Port 8082

Request 4 →

product-instance-1

Port 8081

Conclusion

- ✓ 3 microservices built with Spring Boot 3.2 and Java 21
- ✓ Service discovery via Netflix Eureka — dynamic instance registration
- ✓ Load balancing through Spring Cloud Gateway with lb:// routing
- ✓ All services containerized with Docker and orchestrated via Docker Compose
- ✓ Demonstrated successful traffic distribution across 3 product instances